

INTERNATIONAL UNIVERSITY
Vietnam National University – Ho Chi Minh City



REPORT

Web Application Laboratory

Nguyễn Xuân An – ITITIU22001

LAB 5 EXERCISES: SERVLET & MVC PATTERN

1. Ex5: Search Functionality

Goal: Allow searching for students by Name or Student Code.

 **Student Management System**
MVC Pattern with Jakarta EE & JSTL

-- All Majors -- Search Reset

+ Add New Student

ID	STUDENT CODE	FULL NAME	EMAIL	MAJOR	ACTIONS
5	SV005	David Wilson	david.w@email.com	Computer Science	Edit Delete

1

DAO Layer (StudentDAO.java)

The static SQL statement (`SELECT * FROM students`) is modified into a dynamic SQL statement.

```
// Use StringBuilder to append SQL strings efficiently
StringBuilder sql = new StringBuilder("SELECT * FROM students WHERE 1=1");
// Check if there is a search keyword
if (keyword != null && !keyword.trim().isEmpty()) {
    // Add filter condition. Use OR to search in both columns.
    // The ? mark is a placeholder to prevent SQL Injection.
    sql.append(" AND (full_name LIKE ? OR student_code LIKE ?)");
}
```

- WHERE 1=1:** This condition is always true, which allows subsequent **AND ...** conditions to be appended easily without needing to check if it is the first or second condition.
- ? Placeholder:** Strings are never concatenated directly (e.g., `... LIKE ' ' + keyword + ' '`) to prevent SQL Injection attacks.

Controller Layer (StudentController.java)

```
// Get keyword from URL (e.g., student?search=Nguyen)
```

```
String keyword = request.getParameter("search");
// Call DAO
List<Student> students = studentDAO.getAllStudents(keyword, ...);
// Important: Send the keyword back to the View
request.setAttribute("search", keyword);
```

- **Sending back `search`:** The keyword is sent back to the view so that when the webpage reloads, the search box continues to display the keyword the user entered, maintaining the search context.

2. Ex6: Server-Side Validation

Goal: Prevent invalid or missing data from being saved into the database.

Controller Logic (insertStudent / updateStudent)

```
String error = null;
// 1. Check for empty (Required)
if (fullName == null || fullName.trim().isEmpty()) {
    error = "Full Name is required";
}
// 2. Check Email format (Regex)
else if (!email.matches("^[A-Za-z0-9+_.-]+@(.+)$")) {
    error = "Invalid Email format";
}
// 3. Handle if there is an error
if (error != null) {
    // Send error message
    request.setAttribute("error", error);

    // EXTREMELY IMPORTANT: Re-create the Student object from the input data
    Student student = new Student(studentCode, fullName, email, major);
    request.setAttribute("student", student);

    // Return to the form page
    request.getRequestDispatcher("/views/student-form.jsp").forward(request,
response);
    return; // Stop the function immediately, do not run the save to DB code below
}
```

- **Re-creating `student`:** The `student` object is re-created from the input data so that when returning to the form, the data the user entered is preserved. This

prevents the user from having to re-enter all information due to a validation error.

3. Ex7: Sorting

Goal: Enable sorting by clicking on column headers (ascending/descending).

-- All Majors --

Search

Reset

+ Add New Student

ID	STUDENT CODE	FULL NAME ▲	EMAIL	MAJOR	ACTIONS
5	SV005	David Wilson	david.w@email.com	Computer Science	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	Edit Delete

1

-- All Majors --
Search
Reset

+ Add New Student

ID	STUDENT CODE ▲	FULL NAME	EMAIL	MAJOR	ACTIONS
1	SV001	John Smith	john.smith@email.com	Computer Science	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	Edit Delete

1

View (student-list.jsp)

Logic to create smart sort links:

```
<a href="student?action=list&sort=full_name&order=${sortBy == 'full_name' &&
sortOrder == 'ASC' ? 'DESC' : 'ASC'}">
  Full Name
</a>
```

- **Logic:**
 - If the list is currently sorted by this column (`sortBy == 'full_name'`) AND the order is ascending (`ASC`), the next click will switch to descending (`DESC`).
 - Otherwise (if currently DESC or sorting by a different column), the default order is `ASC`.

DAO Security (StudentDAO.java)

```
if (sortBy != null) {
    // Whitelist: Only allow sorting by real columns
    if (sortBy.matches("^(id|student_code|full_name|email|major)$")) {
        sql.append(" ORDER BY ").append(sortBy).append(" ").append(order);
    }
}
```

```

    } else {
        // If user intentionally modifies URL to a strange column (e.g., password),
        default to id
        sql.append(" ORDER BY id DESC");
    }
}

```

- **Security:** Since `ORDER BY` clauses cannot use the `?` placeholder (PreparedStatement), a manual whitelist check is performed to ensure the column name is valid, preventing SQL Injection.

4. Ex8: Pagination

Goal: Display only 5 students per page.

-- All Majors --
Search
Reset

+ Add New Student

ID	STUDENT CODE	FULL NAME	EMAIL	MAJOR	ACTIONS
6	SV006	Nguyễn Xuân An	annguyenxuan2004@gmail.com	Information Technology	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	Edit Delete

1
2
Next



Student Management System

MVC Pattern with Jakarta EE & JSTL

-- All Majors --

Search

Reset

+ Add New Student

ID	STUDENT CODE	FULL NAME	EMAIL	MAJOR	ACTIONS
1	SV001	John Smith	john.smith@email.com	Computer Science	Edit Delete

Previous

1

2

Mathematical Formula

- limit**: Number of items to display (e.g., 5).
- page**: Current page (e.g., page 2).
- offset**: Starting position in the Database.
 - Page 1: Starts from 0.
 - Page 2: Starts from 5.
 - Page 3: Starts from 10.
 - Formula**: $\text{offset} = (\text{page} - 1) * \text{limit}$

DAO (StudentDAO.java)

```
// MySQL supports LIMIT and OFFSET
sql.append(" LIMIT ? OFFSET ?");
```

Controller - Calculate Total Pages

```
int totalStudents = studentDAO.countStudents(...); // Count total records satisfying
search conditions
// Formula to calculate total pages (round up)
// Example: 13 students, 5 students/page -> 13/5 = 2.6 -> Round up to 3 pages.
int totalPages = (int) Math.ceil((double) totalStudents / limit);
```

View - Maintain State

When changing pages, the current filters (Search, Sort) must be preserved.

```
<a href="student?action=list&page=${i}&search=${search}&sort=${sortBy}...">
    ${i}
</a>
```

- Pagination links include all parameters `search`, `sort`, and `major` to ensure the user remains in the current filter context when navigating to a new page.